Syntaxe des LSTMs

Structure des fichiers lstm et utilisation des formules

État de référence : 6 novembre 2026

Ce document définit la structure des modèles LSTM, la syntaxe des mots et l'utilisation des formules dans les règles. Il décrit leur sémantique sans imposer de moteur d'exécution, de bibliothèque graphique ni d'interface de lancement. La grammaire complète du langage de formules est décrite dans la référence **Formulas** (`syntax.pdf`, source `syntax.typ`), disponible dans le dépôt du langage de formules. Elle précise les types, les opérateurs et leur priorité, les fonctions, les liaisons `let`, les collections et les fonctions prédéfinies.

Les mots **doit** et **interdit** expriment une condition de conformité du modèle. Le mot **recommandé** exprime une convention de rédaction.

Table des matières

1 Organisation du langage 3
1.1 Conventions grammaticales 3
2 Structure du document JSON5 3
2.1 Enveloppe et types 3
2.2 Définition d'un symbole 4
2.3 Définition d'une règle 4
3 Syntaxe des mots et des arguments 5
3.1 Tokens et mots 5
3.2 Arguments nommés et virgules 5
3.3 Deux niveaux d'échappement 6
4 Sémantique des paramètres et de la réécriture 6
4.1 Actualisation d'un token 6
4.2 Correspondance d'un motif 7
4.3 Règles sensibles au contexte 7
4.4 Conditions supplémentaires 8
4.5 Spécificité et choix pondéré 8
4.6 Générations et parallélisme logique 8
5 Expansion graphique et commandes de dessin 9
5.1 Application des règles de dessin 9
5.2 Primitives 9
5.3 Disques, feuilles et fleurs 10
5.4 Remplissage 10
6 Fonctions propres aux LSTM 10
6.1 Fonctions disponibles dans les règles de croissance et de dessin 10
6.2 Lecture du contexte dans les deux familles de règles 11
7 Exemple complet : arbre à fleurs 11

1 Organisation du langage

Un modèle met en œuvre trois niveaux syntaxiques distincts.

1. Le **document JSON5** contient les métadonnées, l'axiome, les définitions de symboles et les règles de croissance et de dessin.
2. Un **mot** du système de Lindenmayer est une chaîne contenant une suite ordonnée de tokens. Un token associe un symbole à des arguments nommés.
3. Une **formule** calcule la valeur d'un argument, une contrainte de règle ou une valeur intermédiaire. Elle possède sa propre grammaire.

Par exemple, dans le champ JSON5 suivant, les guillemets extérieurs délimitent une chaîne JSON5, `F` est un symbole, `length` et `width` sont des noms d'arguments, et `max(2, longueur * 0.8)` est une formule.

```
{ to: "F{length: max(2, longueur * 0.8), width: 2}" }
```

Une formule n'est évaluée que lorsqu'elle occupe un emplacement prévu à cet effet. Le champ `name`, par exemple, demeure du texte. Écrire une formule entre guillemets **à l'intérieur** d'un argument en fait également du texte : `F{length: '2 + 3'}` ne calcule pas une longueur de cinq unités.

1.1 Conventions grammaticales

Les productions présentées ci-dessous utilisent une notation EBNF : `::=` introduit une définition, `|` une alternative, `[ ... ]` une partie facultative et `{ ... }` une répétition de zéro à plusieurs occurrences. Les terminaux figurent entre guillemets. Les espaces entre termes de la grammaire servent à la présentation et ne représentent pas nécessairement des espaces dans la source. Le terminal `ESPACE` représente un caractère d'espacement.

Les signatures de fonctions utilisent une autre convention : dans `f(x[, y])`, les crochets indiquent un argument facultatif et ne doivent pas être recopiés. `x...` désigne un nombre variable d'arguments. Dans une formule effective, les crochets délimitent toujours une liste, un motif de liste ou une opération d'indexation.

2 Structure du document JSON5

2.1 Enveloppe et types

Le document doit être un objet JSON5, mais utilise généralement l'extension `.lstm`.

Champ	Type	Défaut	Signification
`name`	`str`	`""`	Nom affichable du modèle.
`recommended_iterations`	`int`	`5`	Nombre de réécritures recommandé après l'axiome, positif ou nul pour une séquence finie.
`axiom`	`str`	`""`	Mot initial, avant toute réécriture.
`symbols`	`list`	`[]`	Déclarations sous forme de chaînes, avec arguments par défaut facultatifs.
`rules`	`list`	`[]`	Règles de croissance.
`drawing`	`list`	`[]`	Règles de dessin récursives.
`globals`	`object`	`{}`	Variables et fonctions globales évaluées une seule fois à la création du modèle, dans l'ordre de déclaration. Le corps d'une fonction est évalué à chaque appel.
`bg`	entier ou chaîne de formule	`"#FFFFFF"`	Couleur de fond RGB, entre `0x000000` et `0xFFFFFF`. La formule doit produire un entier.
`width`	entier ou chaîne de formule	`1000`	Largeur du canevas en pixels. La valeur doit être un entier strictement positif.

Champ	Type	Défaut	Signification
`height`	entier ou chaîne de formule	`800`	Hauteur du canevas en pixels. La valeur doit être un entier strictement positif.
`seed`	entier ou chaîne	absent	Graine des choix aléatoires. Une chaîne désigne une graine textuelle, sans être évaluée comme une formule.

Ces champs sont facultatifs ; il est recommandé de les écrire explicitement. Un document `{}` représente donc un modèle vide. Les champs supplémentaires à la racine, tels que `description` ou `notes`, sont des métadonnées sans effet sur les règles du modèle.

Les types et les domaines énoncés dans ce document constituent le contrat de rédaction. Chaque valeur doit respecter le type et le domaine de son champ. Les nombres spéciaux admis par JSON5, notamment `NaN` et `Infinity`, ne sont pas des poids, des dimensions ni des couleurs conformes. Les valeurs spéciales du langage de formules, telles que `nan()` et `inf()`, suivent sa propre référence et ne dispensent pas de respecter le domaine du champ concerné.

Dans les valeurs de `globals` et de `locals`, ainsi que dans les champs `cond`, `bg`, `width` et `height`, une chaîne contient une formule ; une valeur littérale peut être écrite directement, par exemple `locals: {pas: 2}` ou `cond: true`. Une chaîne littérale doit donc être citée à l'intérieur de la formule, par exemple `globals: {titre: "'Arbre'"}`. Les dimensions et la couleur de fond sont évaluées après les variables globales.

```
{
  globals: {unite: 200, fond: "#F0F4FF"},
  width: "4 * unite",
  height: 600,
  bg: "fond",
  seed: "arbre-printemps",
}
```

Cet exemple décrit un canevas de 800 × 600 pixels. `fond` est évalué une seule fois, puis sa valeur sert de couleur de fond. `seed` est une valeur littérale, sans accès aux variables globales. Une graine fixe permet de répéter les choix aléatoires à modèle et conditions d'évaluation identiques ; elle ne définit pas une suite universelle commune à tous les moteurs. Son absence ne demande aucune suite reproductible particulière.

2.2 Définition d'un symbole

Chaque entrée de `symbols` est une chaîne décrivant un token et ses valeurs par défaut. Dans `symbols: ["A{x: 2}"]`, les occurrences `A` et `A{x: 5}` reçoivent respectivement 2 et 5. Un symbole sans déclaration reste utilisable et ne possède aucun argument par défaut. Les primitives de dessin sont réservées.

Une déclaration contient uniquement le symbole et ses arguments par défaut. Les objets contenant `symbol`, `drawing` ou `locals` ne sont pas des déclarations valides. Le dessin d'un symbole se définit dans une règle de la liste `drawing`, avec ses éventuelles variables `locals`.

2.3 Définition d'une règle

Les listes `rules` et `drawing` utilisent le même format.

Champ	Présence	Contenu
`from`	obligatoire	Token central, éventuellement entouré de motifs de contexte.
`to`	facultatif	Mot de remplacement ; défaut : `""`.
`weight`	facultatif	Poids numérique fini et positif ou nul ; défaut : `1`.
`name`	facultatif	Nom descriptif de la règle, sans effet sur sa sélection.
`cond`	facultatif	Booléen ou chaîne de formule retournant un booléen, testé après les motifs. Sans ce champ, aucune condition supplémentaire.

Champ	Présence	Contenu
`specificity`	facultatif	Entier remplaçant la spécificité calculée.
`locals`	facultatif	Objet associant des noms à des formules ou à des valeurs littérales. Défaut : `{}`.

Les contextes s'écrivent dans `from`, avec `<` et `>`.

```
{
  from: "A{age: fn x -> x > 0}",
  to: "F{length: longueur}A{age: age - 1, longueur: longueur * 0.8}",
  weight: 3,
}
```

Le motif central porte sur un seul token. Un `to` vide produit le mot vide. Après la sélection d'une règle, ses variables `locals` sont évaluées une fois dans l'ordre de déclaration. Chaque formule voit les arguments du token, les fonctions de contexte et les variables locales précédentes. Le remplacement voit toutes ces valeurs. Les motifs et `cond` ne voient pas les variables locales.

Les arguments d'un token dans `from` sont des contraintes de correspondance, et non des valeurs par défaut. Une contrainte peut tester la présence d'un argument, comparer une valeur ou appeler un prédicat ; ces trois cas sont détaillés dans la section « Correspondance d'un motif ». Les arguments du token testé appartiennent au contexte d'évaluation du motif, sans devenir des variables globales du modèle.

3 Syntaxe des mots et des arguments

3.1 Tokens et mots

```
mot       ::= { ESPACE | token } ;
token     ::= symbole [ "{" [ arguments ] "}" ] ;
arguments ::= argument { "," argument } [ "," ] ;
argument  ::= identificateur ":" expression ;
```

Un symbole est un **caractère Unicode unique**, sensible à la casse. Pour éviter toute ambiguïté, un symbole conforme ne doit être ni un espace ni l'un des délimiteurs `{`, `}`, `:`, `,`, `'` ou `"`, ni l'un des caractères réservés aux motifs de contexte : `+`, `.`, `-`, `*`, `^`, `$`, `|`, `?`, `>`, `<`. Les caractères `[` et `]` sont autorisés comme symboles autonomes et servent aux embranchements du dessin et du contexte. Une lettre accentuée composée de plusieurs points de code ne constitue pas nécessairement un symbole unique.

Les tokens sont juxtaposés, avec ou sans espaces entre eux : `ABC` est le mot de trois tokens `A`, `B`, `C`. Il ne désigne pas un symbole nommé `ABC`. Le bloc d'arguments doit suivre immédiatement le symbole : écrire `A{x: 2}`, et non `A {x: 2}` ni `A:{x: 2}`.

```
A
A{}
A{x: 2, y: 3,}
F{length: 20}[R{deg: 25}F{length: 10}]F{length: 15}
A{donnees: {'nom': 'branche, droite', 'valeurs': [1, 2, 3]}}
```

Dans un mot, les crochets extérieurs sont des tokens. Dans une expression, les crochets appartiennent à la grammaire des formules. Les accolades imbriquées et les délimiteurs présents dans une chaîne de caractères sont reconnus par le découpeur. Une accolade ouvrante d'arguments doit posséder son accolade fermante correspondante.

3.2 Arguments nommés et virgules

Les noms d'arguments suivent la règle des identificateurs du langage de formules. Le séparateur entre nom et valeur est `:`, tandis que `=` est réservé aux liaisons `let`.

Chaque valeur est une expression complète.

```
A{x: let a = 2, b = 3 for a + b, y: 9}
A{f: fn x, y -> x + y, couleur: #FFFFFF}
A{x: [1, 2, 3].sum(), y: {'a': 4, 'b': 5}.a}
```

Dans le premier exemple, `x` vaut `5` et `y` vaut `9`. Dans le second, la fonction possède deux paramètres, puis `couleur` commence un nouvel argument du token.

Les arguments devraient posséder des noms distincts. En cas de répétition d'un nom, seule la dernière expression est conservée ; les expressions précédentes de ce nom ne sont pas évaluées.

3.3 Deux niveaux d'échappement

Le lecteur JSON5 décode d'abord la chaîne contenant le mot. Le lecteur des formules décode ensuite les chaînes présentes dans les arguments. Il faut donc distinguer les échappements destinés à chacun des lecteurs.

```
{
  axiom: "A{texte: 'branche', message: 'ligne 1\\nligne 2'}",
  symbols: ["A"],
}
```

Dans le fichier ci-dessus, `\\n` devient les deux caractères `\n` après lecture JSON5 ; le lecteur des formules en fait ensuite un saut de ligne. Les commentaires JSON5 doivent rester à l'extérieur des chaînes contenant les mots. Une formule ne possède pas de syntaxe de commentaire ; `//` y est l'opérateur de division entière et `#` y introduit une couleur.

4 Sémantique des paramètres et de la réécriture

4.1 Actualisation d'un token

L'**actualisation** remplace les expressions d'un token par leurs valeurs. Soient D les arguments par défaut du symbole, E les arguments explicites du token et C le contexte fourni par l'opération en cours. On construit la table D complétée par E, en donnant priorité à E pour les noms communs. Chaque expression de cette table est ensuite évaluée dans le même contexte C.

Les résultats des arguments voisins ne sont pas ajoutés à C. Les arguments ne forment donc pas une suite de liaisons locales.

```
A{x: 2, y: x + 1}
```

Si le contexte contient `x = 10`, ce token reçoit `x = 2` et `y = 11`. Si le contexte ne contient pas `x`, l'évaluation de `y` échoue. Pour exprimer un calcul local séquentiel, utiliser `let`, ou répéter une formule commune lorsque cela ne modifie pas le sens du calcul.

Le contexte est déterminé par la phase du traitement :

Phase	Variables disponibles
Axiome	Contexte utilisateur vide ; fonctions et variables globales disponibles. Chaque token est indépendant.
Remplacement d'une règle	Arguments du token remplacé et variables locales de la règle sélectionnée.
Expansion graphique	Arguments du token courant et variables locales de sa règle. Un enfant reçoit une valeur de sa règle parente par ses arguments.

Les arguments du prédécesseur ne sont pas recopiés automatiquement dans les successeurs. Par exemple, pour conserver `age` et `couleur`, il faut les transmettre explicitement, ou prévoir des valeurs par défaut appropriées.

Un défaut `x: x` peut reprendre un `x` du contexte appelant, mais échoue dans l'axiome si aucun `x` global n'est défini.

L'axiome peut contenir des formules : `A{x: 2 + 3}` est valide. Elles sont évaluées lors de la première image. Une formule aléatoire placée dans l'axiome est donc tirée à ce moment, puis sa valeur est conservée dans l'axiome actualisé.

4.2 Correspondance d'un motif

Un motif correspond à un token si et seulement si leurs symboles sont identiques et si chacune des contraintes nommées du motif est satisfaite. Un argument exigé mais absent du token fait échouer la correspondance. Les arguments supplémentaires du token ne constituent pas un obstacle. Les valeurs par défaut du symbole ne sont pas ajoutées au motif `from`.

La contrainte `nom: _` est un **joker de présence** : elle accepte toute valeur de l'argument `nom`, y compris `null`, mais exige que cet argument existe. Elle ne résout pas de variable `_` et ne crée aucune liaison. Elle compte comme une contrainte dans le calcul de la spécificité, comme les autres arguments explicitement présents dans le motif.

Pour toute autre contrainte `nom: expression`, l'expression est évaluée dans le contexte des arguments du token testé.

1. Si le résultat n'est pas une fonction, il est comparé par égalité à la valeur de l'argument `nom` du token.
2. Si le résultat est une fonction, elle reçoit la valeur de l'argument testé et doit retourner un booléen. La contrainte est satisfaite uniquement si ce booléen vaut `true`. Une fonction définie avec `fn` doit déclarer exactement un paramètre de premier niveau.

```
A                              correspond à tout token de symbole A
A{age: _}                      exige la présence de age, quelle que soit sa valeur
A{age: 0}                      exige age égal à zéro
A{age: fn x -> x > 0}           exige un âge strictement positif
A{taille: limite}               compare taille à l'argument limite
A{age: fn x -> x > 0, type: 2}  impose les deux contraintes
```

Un nom libre n'introduit pas une variable de capture : `A{x: v}` recherche la valeur de `v` dans le contexte ; il ne lie pas `v` à la valeur de `x`. Le prédicat `fn x -> x` est incorrect pour un argument numérique : retourner `1` ne remplace pas le retour booléen `true` exigé ici. Un motif de liste tel que `fn [a, b] -> a < b` constitue un seul paramètre et est admissible si la valeur testée est une liste de longueur deux. Il peut ignorer un élément : `fn [_, b] -> b > 0`. Les paramètres ignorés continuent de compter dans l'arité : `fn _, _ -> true` possède deux paramètres et ne constitue donc pas un prédicat de règle valide.

Le joker de présence doit être l'expression entière de la contrainte. `A{x: '_'}` compare avec une chaîne littérale, tandis que `A{x: [_]}` évalue une liste contenant une référence au nom `_` ; cette liste n'est pas un motif de déstructuration. Le nom d'argument lui-même n'est pas un joker : `A{_: 2}` teste un argument nommé `_`. Le joker ne change ni le symbole recherché ni le contexte transmis au remplacement : après `A{x: _}`, la formule `x` reste disponible dans `to`, mais aucune variable `_` n'est ajoutée.

Une fonction prédéfinie peut également servir de prédicat si elle accepte l'argument testé et retourne un booléen. L'égalité suit les règles du langage de formules, notamment `1 == true`, et ne constitue pas un contrôle strict du type.

Par exemple, `A{actif: bool}` applique la conversion booléenne à la valeur de `actif`. Cela ne signifie pas qu'un prédicat peut retourner directement un nombre : `A{actif: fn x -> 1}` reste invalide.

4.3 Règles sensibles au contexte

Il est possible de définir des règles qui ne s'appliquent que dans certains contextes, à gauche ou à droite du token cible. Ces contextes sont spécifiés par des motifs de tokens entourant le token principal. Ces motifs ont également des règles spéciales.

- `A` match avec le token `A`

- `A{arg: val}` match avec le token `A` ayant un argument `arg` de valeur `val`. Le joker _ peut être utilisé pour indiquer la présence d'un argument sans se soucier de sa valeur. Une formule peut également être utilisée (l'environement de la formule contient les arguments du token observé)
- `AB` match avec la séquence de tokens `A` suivie de `B`
- `A?` match avec le token `A` de manière optionnelle, c'est-à-dire que la règle peut s'appliquer même si le token `A` n'est pas présent dans le contexte.
- `A*` match avec zéro ou plusieurs occurrences du token `A`
- `A+` match avec une ou plusieurs occurrences du token `A`
- `A[n]` match avec exactement `n` occurrences du token `A`
- `A[m,n]` match avec au moins `m` et au plus `n` occurrences du token `A`
- `A|B` match avec le token `A` OU le token `B`
- `(...)` groupage de règles
- `.` match avec n'importe quel token
- `^` match avec le début du contexte (à gauche)
- `$` match avec la fin d'une branche du contexte (à droite)

Une règle de croissance ou de dessin devient sensible au contexte grâce aux symboles `<` et `>`, qui indiquent respectivement le contexte à gauche et à droite du token principal.

```
from: "A < B > Z"
```

match avec le token `B`, s'il est *immédiatement précédé* du token `A` et *immédiatement suivi* du token `Z` dans le contexte.

Le contexte gauche remonte la chaîne des ancêtres et ignore les branches parallèles. Le contexte droit peut suivre chaque branche issue du token et s'arrête à sa fermeture.

4.4 Conditions supplémentaires

Il est possible d'ajouter une condition supplémentaire à une règle avec le champ `cond`. Sa valeur doit être un booléen ou une formule retournant un booléen. La règle s'applique uniquement si cette valeur est `true` ; une valeur numérique telle que `1` ne remplace pas un booléen.

Dans l'environement d'évaluation de cette formule se trouve les arguments du token observé, ainsi que les contextes gauche et droit accessibles via `lctx()` et `rctx()`.

Une condition supplémentaire n'augmente pas la spécificité d'une règle. Elle sert uniquement à restreindre les situations dans lesquelles la règle peut s'appliquer.

4.5 Spécificité et choix pondéré

La spécificité vaut le nombre d'arguments contraints sur le token central, plus un par côté de contexte spécifié. `specificity` peut remplacer ce total. Parmi les règles qui passent les motifs et `cond`, on retient le groupe de spécificité maximale dont le poids total est positif, puis choisit une règle proportionnellement à son poids. Les `locals` de la règle choisie, puis son `to`, sont ensuite évalués.

Une contrainte littérale ne reçoit pas de priorité supplémentaire par rapport à un prédicat. Deux motifs ayant deux contraintes sont aussi spécifiques, même si l'un paraît intuitivement plus restrictif. L'ordre du fichier n'est pas une priorité de type « première règle correspondante ».

Si les règles retenues ont des poids $w_1, \ldots, w_k$ et une somme $W > 0$, la probabilité nominale de choisir la règle i est $\frac{w_i}{W}$. Les poids n'ont pas besoin de totaliser un. Des poids `3` et `1` représentent ainsi les proportions trois quarts et un quart. Les poids sont des nombres JSON5 fixes, et non des chaînes de formules.

Sans règle sélectionnée, la croissance conserve le token et le dessin le traite comme terminal.

4.6 Générations et parallélisme logique

Une génération remplace chaque token du mot courant à partir de ses valeurs avant réécriture. Les successeurs ne sont pas réécrits à leur tour dans cette même génération. Les remplacements sont concaténés en conservant l'ordre des tokens du mot initial. Il s'agit d'une réécriture parallèle au sens des systèmes de Lindenmayer.

L'actualisation et le dessin des successeurs suivent l'ordre du mot, ce qui détermine les observations de l'état de la tortue.

Pour une séquence finie, le nombre de réécritures doit être un entier positif ou nul. La génération 0 est l'axiome actualisé ; après N réécritures, on atteint la génération N. Zéro réécriture conserve donc l'axiome.

5 Expansion graphique et commandes de dessin

5.1 Application des règles de dessin

Pour chaque image, chaque token du mot de cette génération est développé avec `drawing`. Les enfants sont développés de gauche à droite jusqu'aux terminaux. Une primitive terminale est transmise au dessinateur. Tout autre terminal est invisible. Un remplacement graphique n'altère pas le mot de croissance.

Les tokens produits par les règles de dessin ne sont jamais ajoutés au contexte. Chaque descendant graphique consulte les voisins du token de croissance dont il est issu ; seul le motif central porte sur le token graphique courant. Par exemple, dans le mot `ABZ`, si le dessin de `B` produit `XY`, la règle (de dessin) `A < Y > Z` peut s'appliquer, mais pas `X < Y`

`pos()` et `heading()` lisent la tortue au moment de l'évaluation. Si le dessin de `X` avance, celui de `Y` voit cette nouvelle position. Lorsque le mot est encore en construction, lire son contexte droit actualise à l'avance les tokens nécessaires jusqu'à la fin de la branche, à la position actuelle de la tortue. Ces tokens sont mémorisés et seront dessinés à leur tour.

Une règle peut se rappeler avec un argument décroissant. L'expansion graphique doit se terminer ; chaque chemin récursif doit atteindre un terminal. Exemple :

```
{
  axiom: "A{x: 3}",
  symbols: ["A{x: 0}"],
  drawing: [
    {from: "A", cond: "x > 0", locals: {pas: "2*x"},
     to: "F{length: pas}A{x: x-1}"},
  ],
}
```

Le résultat contient trois traits de longueurs 6, 4 et 2. Le dernier `A{x: 0}` est terminal et invisible.

5.2 Primitives

Le repère du canevas est centré, avec l'axe horizontal orienté vers la droite et l'axe vertical vers le haut. Une unité correspond à un pixel. Les tailles de points et les épaisseurs sont exprimées en pixels. Les angles sont exprimés en degrés. Les paramètres omis reçoivent les valeurs de la table suivante. Ces valeurs sont des défauts du **dessinateur**, distincts des défauts de `symbols`.

Ces symboles sont indisponibles pour les définitions dans `symbols` et sont réservés aux primitives de dessin.

Token	Paramètres par défaut	Effet
`F`	`length: 1`, `width: 1`	Avancer en traçant un trait d'épaisseur `width`. Une longueur négative recule.
`C`	`color: 0`	Fixer la couleur du trait et du remplissage à partir d'un entier RGB.
`R`	`deg: 0`	Tourner dans le sens horaire ; une valeur négative tourne dans le sens antihoraire.
`L`	`size: 1`	Tracer un disque de rayon `size` (diamètre `2 * size`), centré à `size` unités devant la position courante, sans modifier la position de la tortue.
`M`	`length: 1`	Avancer sans tracer. Une longueur négative recule.
`P`	`x, y`	Placer la tortue à la position `(x, y)` sans tracer. Si `x` ou `y` est omis, la coordonnée correspondante reste inchangée.

Token	Paramètres par défaut	Effet
`D`	`x, y, width: 1`	Placer la tortue aux coordonnées absolues `(x, y)` en traçant avec un trait de largeur `width`. Si `x` ou `y` est omis, la coordonnée correspondante reste inchangée.
`H`	`rot`	Tourner la tortue à l'orientation `rot`. Si `rot` est omis, l'orientation reste inchangée.
`(`	aucun	Commencer un remplissage
`)`	aucun	Terminer un remplissage
`[`	aucun	Empiler la position, l'orientation et les couleurs courantes.
`]`	aucun	Dépiler et restaurer cet état, sans tracer le déplacement de retour.

Les autres symboles terminaux ne produisent aucun dessin. Les arguments sans usage pour une primitive sont également ignorés. Les largeurs et les rayons doivent être strictement positifs pour un modèle graphique conforme. Une couleur doit être un entier compris entre `0x000000` et `0xFFFFFF`. La notation de formule `#RRGGBB` produit précisément un tel entier.

Les crochets et les parenthèses doivent être équilibrés, et aucun préfixe du mot graphique ne doit contenir plus de fermetures que d'ouvertures. De plus, les parenthèses, bien qu'elles peuvent techniquement être imbriquées, ne le devraient pas sémantiquement dans un mot graphique. Les deux délimitations peuvent se chevaucher.

```
[F(F]F)
```

est un mot valide.

Au début de chaque image, le dessin précédent est effacé ; la tortue est placée au centre inférieur du canevas, orientée vers le haut. La couleur est conservée entre les images ; l'épaisseur est définie par chaque commande.

5.3 Disques, feuilles et fleurs

L'argument `size` du symbole `L` est un **rayon**. Le décalage du centre du disque le place à `size` unités devant la tortue, soit à une distance égale au rayon. Pour obtenir un disque de rayon `r` centré sur la position courante, définir par exemple la macro suivante :

```
{
  symbols: ["Z{r: 3, c: #FFFFFF}"],
  drawing: [{from: "Z", to: "[C{color: c}M{length: -r}L{size: r}]"}],
}
```

Le déplacement de `-r` compense le décalage du centre de `L`. Avec `L{size: 1}`, le disque a donc un rayon de `1`, un diamètre de `2` et un centre situé à `1` unité devant la tortue. Les crochets préservent la position, l'orientation et les couleurs du dessin environnant.

5.4 Remplissage

Les parenthèses `(` et `)` sont utilisées pour délimiter les zones de remplissage dans le mot graphique. Tout ce qui se trouve à l'intérieur d'une paire de parenthèses est considéré comme faisant partie du remplissage.

Dans une zone de remplissage, les commandes fonctionnent comme d'habitude, sauf que l'intérieur de la forme créée par la tortue est rempli de la couleur du pinceau.

6 Fonctions propres aux LSTM

Les fonctions suivantes donnent accès à l'état du modèle, à son contexte et à son dessin depuis les formules.

6.1 Fonctions disponibles dans les règles de croissance et de dessin

`pos()` Retourne la position actuelle de la tortue sous la forme de `[x, y]` où `x` et `y` sont des `float`.

`heading()` Retourne l'orientation actuelle de la tortue en degrés (0 vers la droite, 90 vers le haut), sous forme de `float`.

`mark(val)` Enregistre la valeur dans une liste qui sera disponible pour consultation dans la prochaine itération. Retourne `null`.

`marked()` Retourne la liste des valeurs enregistrées par `mark` lors de l'itération précédente.

`gen()` Retourne le numéro de la génération actuelle du modèle ; l'axiome correspond à la génération 0.

`screen()` Retourne les dimensions du canevas sous la forme `[largeur, hauteur]`, deux entiers en pixels.

Ces fonctions décrivent l'état du dessin pendant l'évaluation d'une génération, indépendamment de l'affichage ou de l'enregistrement de l'image. `marked()` ne contient pas les marques en cours de collecte pour la génération actuelle.

`pos()` et `heading()` décrivent l'état courant de la tortue, y compris dans un motif de contexte : ils ne restituent pas l'état au moment du dessin d'un voisin lors de la génération précédente. Des tokens successifs issus d'une même règle de croissance peuvent donc observer des positions différentes si le dessin du premier déplace la tortue avant l'actualisation du suivant.

6.2 Lecture du contexte dans les deux familles de règles

`lctx([depth])` et `rctx([depth])` Retourne l'arbre de contexte à gauche ou à droite du token. Si `depth` est fourni, retourne une liste des tokens disponible à cette profondeur de l'arbre (Un nombre négatif part des feuilles de l'arbre). Lorsque tout le contexte est retourné, il est sous forme de liste de tokens (chaque token est un dict `{"symbol": ..., "args": {...}}`). Le dernier argument de cette liste peut aussi être une liste au lieu d'un token. Cette liste contient les listes des sous-arbres enfant du contexte.

7 Exemple complet : arbre à fleurs

Le modèle suivant combine paramètres, prédicats, règles pondérées, transmission explicite du contexte et macro de disque. Enregistrer le bloc dans `arbre_fleuri.json5`. Les quatre générations demandées transforment les bourgeons `A` en ramifications ; le dessin final remplace les bourgeons restants par des fleurs.

```
{
  name: "Arbre à fleurs blanches",
  description: "Exemple autonome de réécriture paramétrique pondérée.",
  recommended_iterations: 4,
  axiom: "C{color: #5C4033}A{n: 4, longueur: 48, largeur: 5}",
  symbols: ["A{n: 0, longueur: 12, largeur: 1}", "Z{r: 3, c: #FFFFFF}"],
  drawing: [
    {from: "A", to: "Z{r: 4, c: #FFF8EB}"},
    {from: "Z", to: "[C{color: c}M{length: -r}L{size: r}]"},
  ],
  rules: [
    {
      from: "A{n: fn x -> x > 0}",
      to: "F{length: longueur, width: largeur}\
           [R{deg: -24}\
            A{n: n-1, longueur: longueur*0.76, largeur: max(1, largeur*0.7)}]\
           [R{deg: 28}\
            A{n: n-1, longueur: longueur*0.8, largeur: max(1, largeur*0.7)}]",
      weight: 3,
    },
    {
      from: "A{n: fn x -> x > 0}",
      to: "F{length: longueur, width: largeur}\
           [R{deg: -18}\
            A{n: n-1, longueur: longueur*0.8, largeur: max(1, largeur*0.7)}]\
           [R{deg: 34}\
            A{n: n-1, longueur: longueur*0.7, largeur: max(1, largeur*0.7)}]",
      weight: 1,
    },
```

```
  ],
}
```

Les barres obliques inverses en fin de ligne prolongent une chaîne JSON5 ; elles n'appartiennent pas au mot après décodage. Les espaces d'indentation restants sont sans effet entre les tokens. Il est également possible d'écrire chaque champ `to` sur une seule ligne.

Les deux règles ont une spécificité égale à un, et leurs poids donnent les proportions nominales trois quarts et un quart. Quand `n` vaut zéro, aucune des deux règles ne correspond : le token `A` reste dans le mot. Il n'est transformé en fleur que pendant la phase graphique. Un appel de dessin à une génération antérieure représente également les bourgeons encore actifs par leur macro graphique ; il ne les supprime pas des générations suivantes.